

REPORT

Permutation Generation with Position Restrictions Using Backtracking

1. Title

Permutation Generation with Position Restrictions

2. Aim

To design and implement a **Backtracking algorithm** that generates all permutations of a set of distinct elements while ensuring that elements are not placed in their restricted positions.

3. Problem Statement

Given a set of elements and a list of position restrictions, generate all possible permutations that satisfy the restrictions.

The algorithm must:

- Build permutations one position at a time.
 - Check position constraints before placing an element.
 - Reject invalid assignments using pruning.
 - Avoid duplicate permutations.
 - Generate all valid permutations.
-

4. Input

The program accepts:

- Number of elements n
- A set of distinct elements
- Number of restrictions
- Each restriction consists of:
 - Position
 - Element that cannot occupy that position

Example:

Number of elements = 4

Elements:

A B C D

Restrictions:

1 A

3 C

This means:

- A cannot be at position 1.
 - C cannot be at position 3.
-

5. Output

The program displays all permutations satisfying the specified restrictions.

Example:

B A C D

B A D C

B C A D

B C D A

...

Only valid permutations are displayed.

6. Backtracking Approach

Backtracking constructs a solution incrementally.

At each position, the algorithm:

1. Selects an unused element.
2. Checks whether the element is allowed at the current position.
3. If the element is restricted, it is rejected.
4. Otherwise, the element is placed.
5. The algorithm recursively fills the next position.
6. After returning from recursion, the choice is removed.
7. Another unused element is tried.

This process continues until all positions have been filled.

7. Position Constraint

An element can only be placed if it satisfies the restriction for the current position.

For example:

Position 1 $\rightarrow$ A is restricted

Therefore:

A $\rightarrow$ Position 1 ✖

B $\rightarrow$ Position 1 ✔

C $\rightarrow$ Position 1 ✔

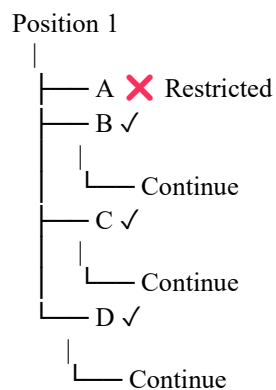
D $\rightarrow$ Position 1 ✔

The invalid choice is rejected immediately.

8. Pruning

Pruning prevents the algorithm from exploring branches that can never produce a valid solution.

For example:



The branch beginning with A is never explored further.

This saves unnecessary computation.

9. Algorithm

```
BACKTRACK(position)

  IF position == n THEN
    PRINT permutation
    RETURN
  END IF

  FOR each element i

    IF element i is already used THEN
      CONTINUE
    END IF

    IF element i is restricted at position THEN
      CONTINUE
    END IF

    permutation[position] = element i
    used[i] = TRUE

    BACKTRACK(position + 1)

    used[i] = FALSE

  END FOR

END BACKTRACK
```

10. Pseudocode

```
ALGORITHM GeneratePermutations(elements, n, restrictions)

BEGIN

  FOR i = 0 TO n-1
    used[i] = FALSE
  END FOR

  Backtrack(0)

END

PROCEDURE Backtrack(position)

BEGIN

  IF position = n THEN
    PRINT permutation
    RETURN
  
```

```
END IF

FOR i = 0 TO n-1 DO

    IF used[i] = TRUE THEN
        CONTINUE
    END IF

    IF IsRestricted(elements[i], position) = TRUE THEN
        CONTINUE
    END IF

    permutation[position] = elements[i]
    used[i] = TRUE

    Backtrack(position + 1)

    used[i] = FALSE

END FOR

END
```

11. Example

Consider:

Elements = {A, B, C, D}

Restrictions:

Position 1 $\rightarrow$ A

Position 3 $\rightarrow$ C

The algorithm first tries to fill position 1.

A is rejected because it is restricted.

The algorithm can therefore try:

B
C
D

Suppose it selects:

B

The partial permutation becomes:

B _ _ _

The algorithm then proceeds to position 2 and continues recursively.

If it eventually encounters:

B A C _

then C cannot be placed at position 3, so that branch is rejected immediately.

12. Avoiding Duplicate Permutations

The `used[]` array records whether an element has already been placed.

For example:

A B _ _

At this point:

`used[A] = TRUE`

`used[B] = TRUE`

Therefore, A and B cannot be selected again until the algorithm backtracks.

This ensures that each element occurs only once in a permutation.

Since the input is assumed to contain **distinct elements**, duplicate permutations are avoided.

13. Correctness

The algorithm is correct for the following reasons:

Every Position Is Filled

The recursion continues until:

`position == n`

At that point, all positions contain an element.

Restrictions Are Satisfied

An element is placed only after checking its position restriction.

Therefore, no printed permutation contains a prohibited placement.

No Element Is Repeated

The `used[]` array prevents an element from being selected more than once in the same permutation.

All Valid Solutions Are Considered

For each position, every unused and allowed element is considered.

Therefore, every valid permutation can be generated.

Hence, the algorithm produces **all and only valid permutations**.

14. Complexity Analysis

For n distinct elements, there can be:

$$n!$$

possible permutations without restrictions.

Therefore, the search space grows factorially.

Time Complexity

A simple upper-bound analysis is:

$$O(n \times n!)$$

The additional factor comes from trying candidates at each level.

If restrictions are checked using a direct lookup table, each restriction check takes:

$$O(1)$$

Space Complexity

The algorithm uses:

- `permutation[]` $\rightarrow O(n)$
- `used[]` $\rightarrow O(n)$
- Recursion stack $\rightarrow O(n)$

Therefore:

$$O(n)$$

auxiliary space, excluding the space needed to store/print the output.

15. Effect of Restrictions

Restrictions reduce the number of feasible permutations.

Without restrictions:

$$n!$$

possible permutations exist.

With restrictions, some branches are eliminated before complete permutations are generated.

Therefore, stronger restrictions can significantly reduce the actual search performed by the algorithm.

However, the worst-case complexity remains factorial because there may be few or no restrictions.

16. Advantages

- Generates only valid permutations.
- Performs constraint checking during construction.
- Uses pruning to avoid invalid branches.
- Prevents repeated use of elements.

- Complete for distinct input elements.
 - Easy to implement using recursion.
-

17. Limitations

- Factorial growth makes the algorithm expensive for large n .
 - The number of possible permutations can become extremely large.
 - Backtracking improves practical performance but does not eliminate the factorial worst case.
-

18. Applications

Position-restricted permutation generation can be used in:

- Scheduling problems
 - Assignment problems
 - Seating arrangements
 - Task allocation
 - Puzzle solving
 - Constraint satisfaction problems
 - Resource allocation
-

19. Sample Input

```
4
A B C D
2
1 A
3 C
```

20. Sample Output

Valid Permutations:

```
-----
B A C D
B A D C
B C A D
```

B C D A
B D A C
B D C A
C A B D
C A D B
C B A D
C B D A
C D A B
C D B A
D A B C
D A C B
D B A C
D B C A
D C A B
D C B A

Use the exact output produced by your own program in your report screenshot.

21. Result

The permutation generation problem was successfully solved using a **Backtracking algorithm**.

The algorithm constructs permutations one position at a time, checks restrictions before placing elements, and backtracks whenever necessary.

The use of pruning prevents invalid assignments from being explored further.

22. Conclusion

Backtracking provides an effective approach for generating permutations with position restrictions. By checking constraints before recursive branching, invalid solutions are eliminated early.

The algorithm is complete and ensures that every valid permutation is generated without repeating elements.

Although pruning can substantially improve practical performance, permutation generation still has factorial worst-case complexity:

$$O(n \times n!)$$

Therefore, the approach is most suitable for small and moderate input sizes.